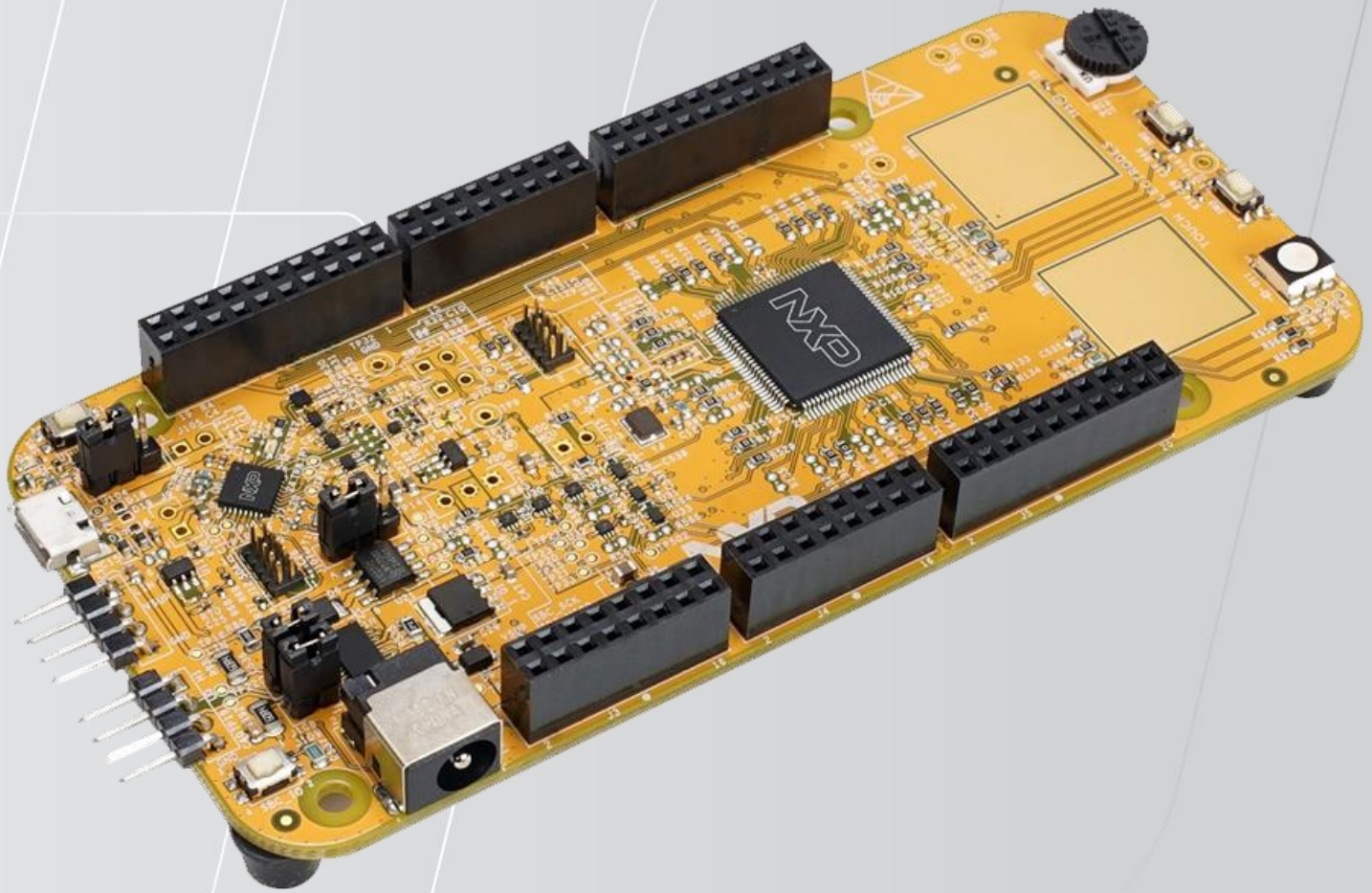
StartUp Code



16 Oct 2023

Outline

1. Memory
   + ROM
   + RAM

2. Linker Script

3. Vector table

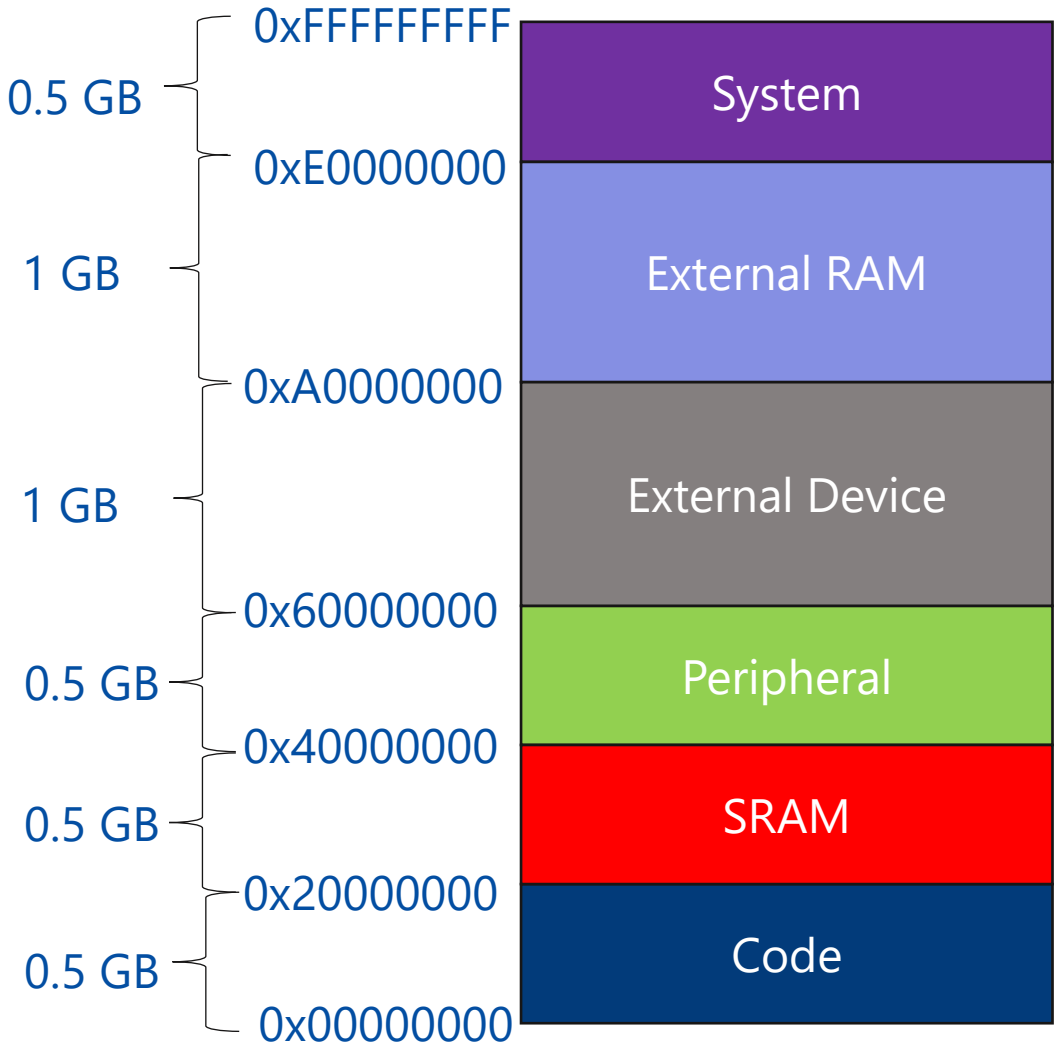
4. Mapping Code and Data to the Target

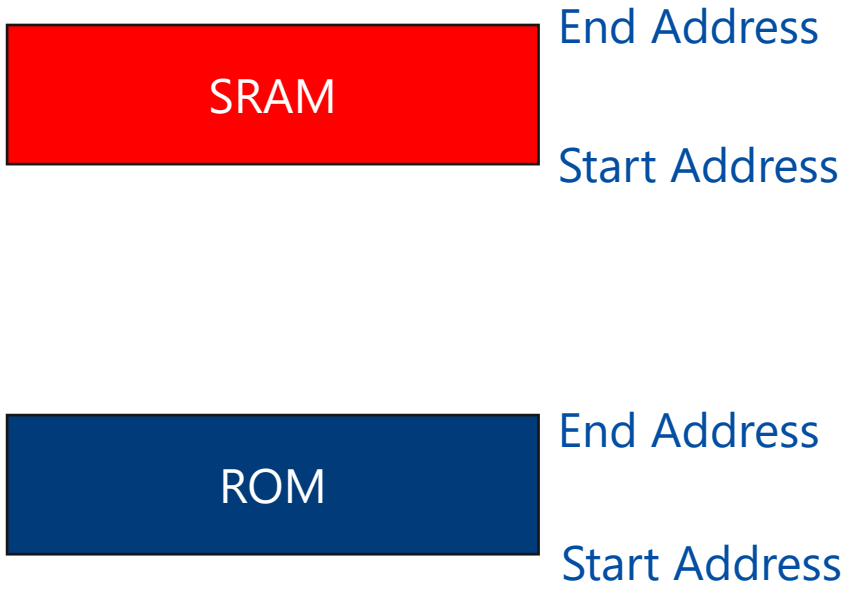
5. Compiler-specific Function, Variable, and Type Attributes

6. Booting process

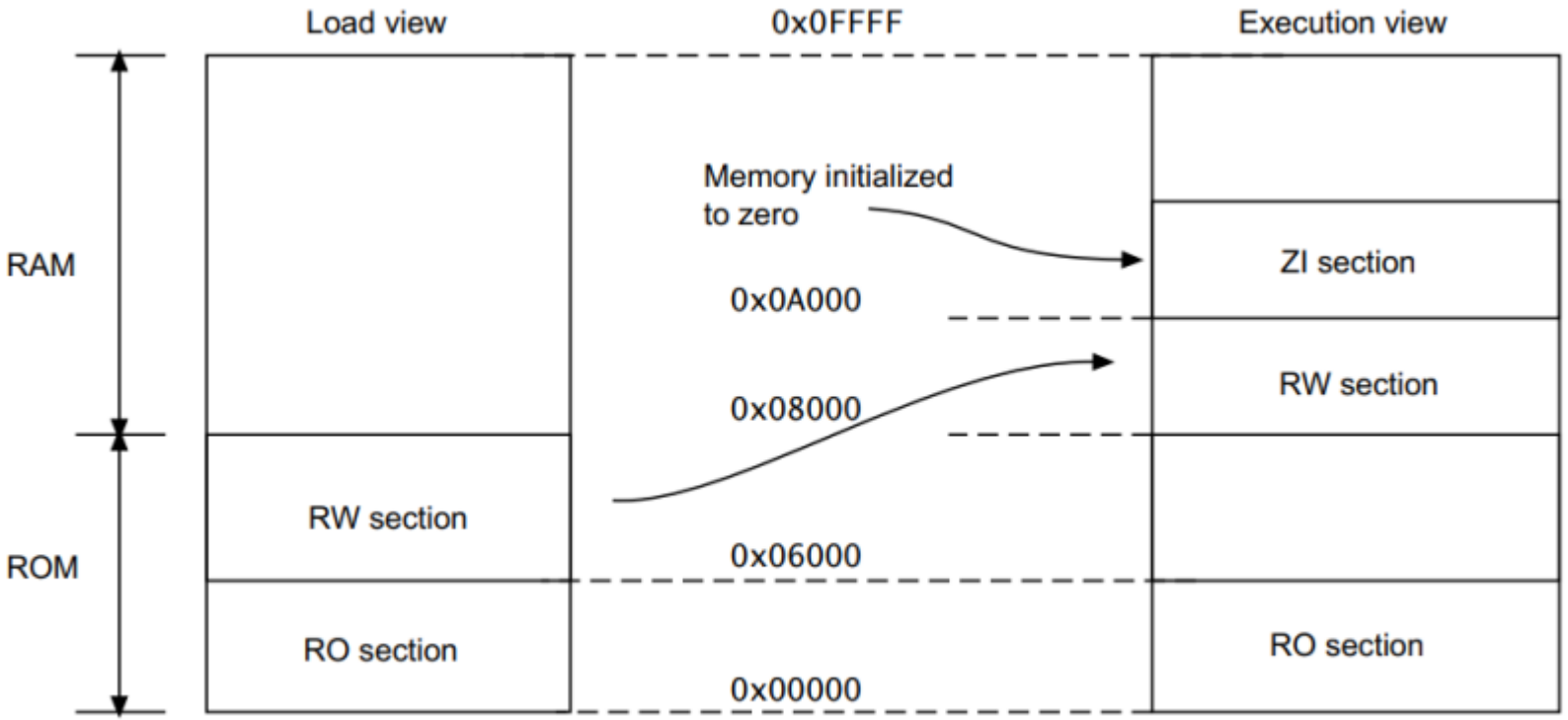
7. Inline assembly code

1. Memory

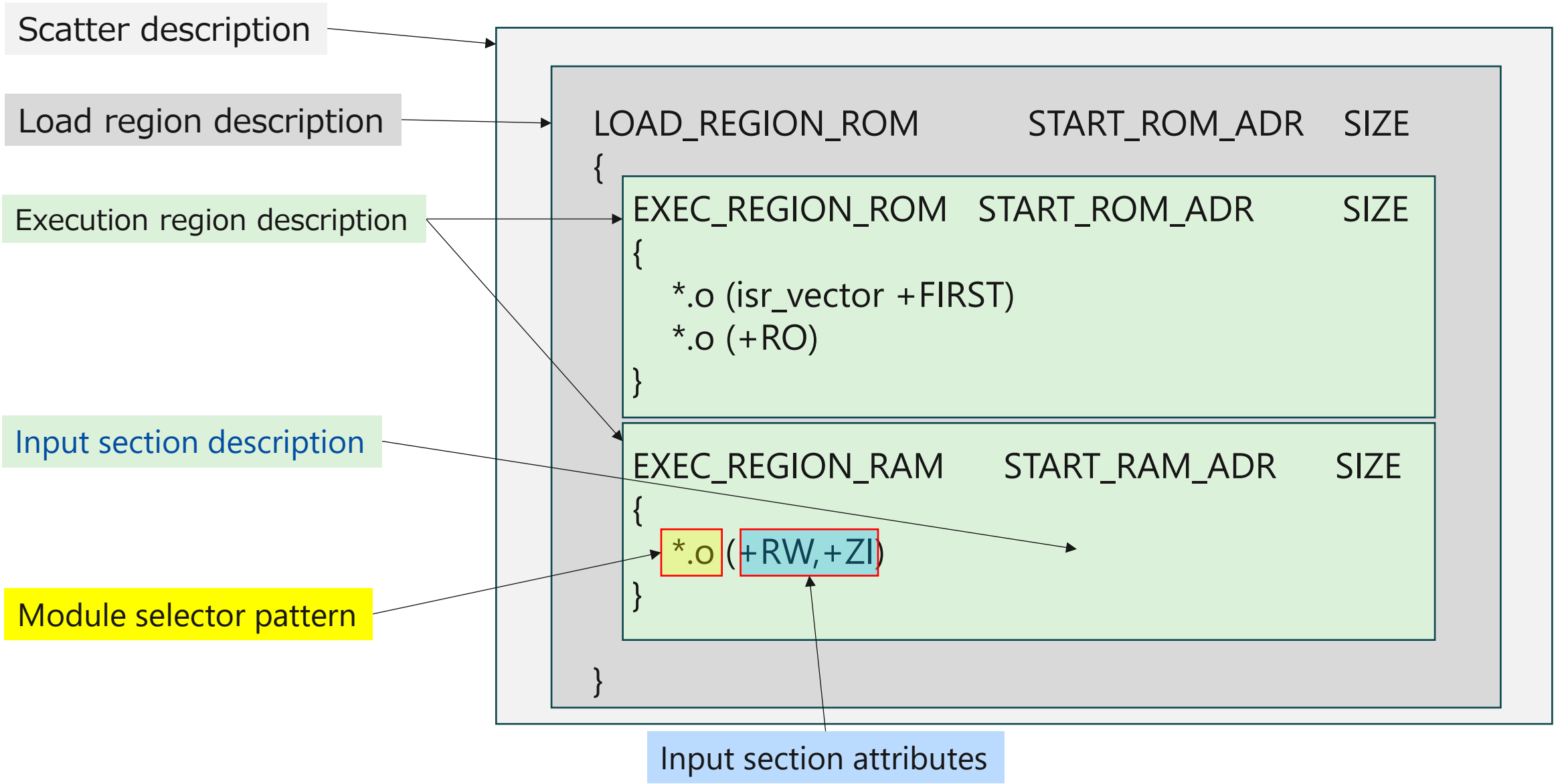

0xFFFFFFFF
0.5 GB
System
0xE0000000
1 GB
External RAM
0xA0000000
1 GB
External Device
0x60000000
0.5 GB
Peripheral
0x40000000
0.5 GB
SRAM
0x20000000
0.5 GB
Code
0x00000000


SRAM
End Address
Start Address
ROM
End Address
Start Address

1. Memory


Load view
0x0FFFF
Execution view
RAM
ROM
Memory initialized
to zero
0x0A000
0x08000
0x06000
0x00000
ZI section
RW section
RW section
RO section
RO section

2. Linker Script

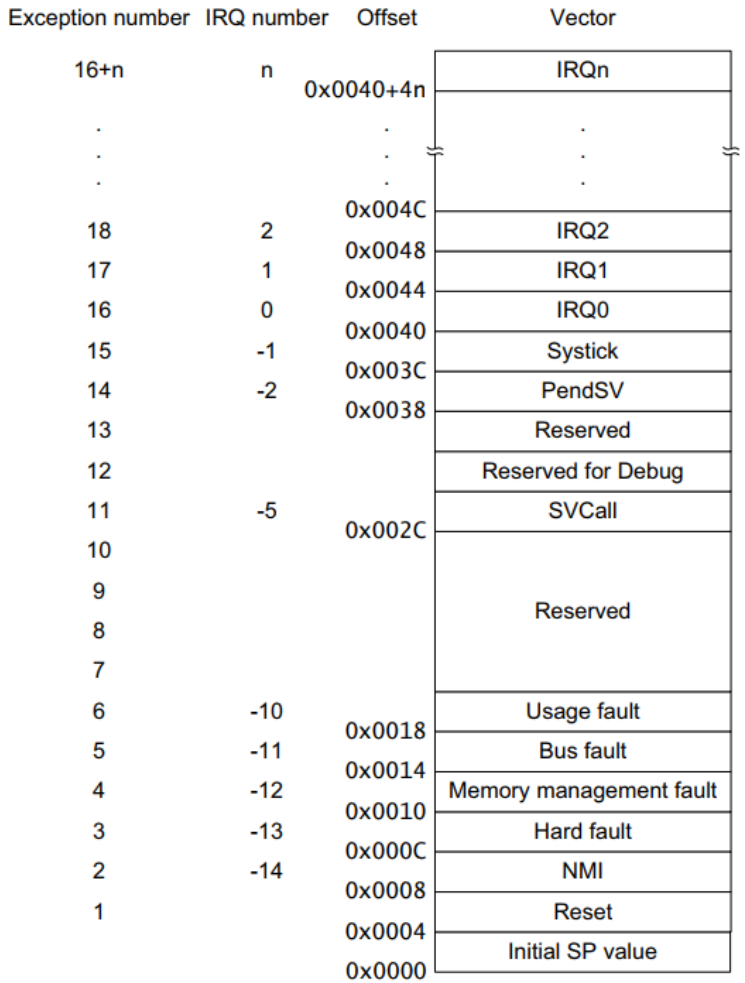


3. Vector table

To create a vector table for interrupts using an array and map it into a specific section (isr_vector):

```
static const unsigned int vector_table[]
__attribute__( (used, section("isr_vector") )) =
{
    (unsigned int)0x20004000,
    (unsigned int)&Reset_Handler
};
```

Vector Table Array: The vector_table array is defined to hold pointers to the ISRs. The first entry is the initial stack pointer, followed by the address of each ISR.



4. Mapping Code and Data to the Target

C1: Using __attribute__

__attribute__((section("*name*")))

C2: Using #pragma clang section

#pragma clang section [*section_type_list*]

5. Compiler-specific

Function attributes:

__attribute__((alias))

__attribute__((section("*name*")))

__attribute__((used))

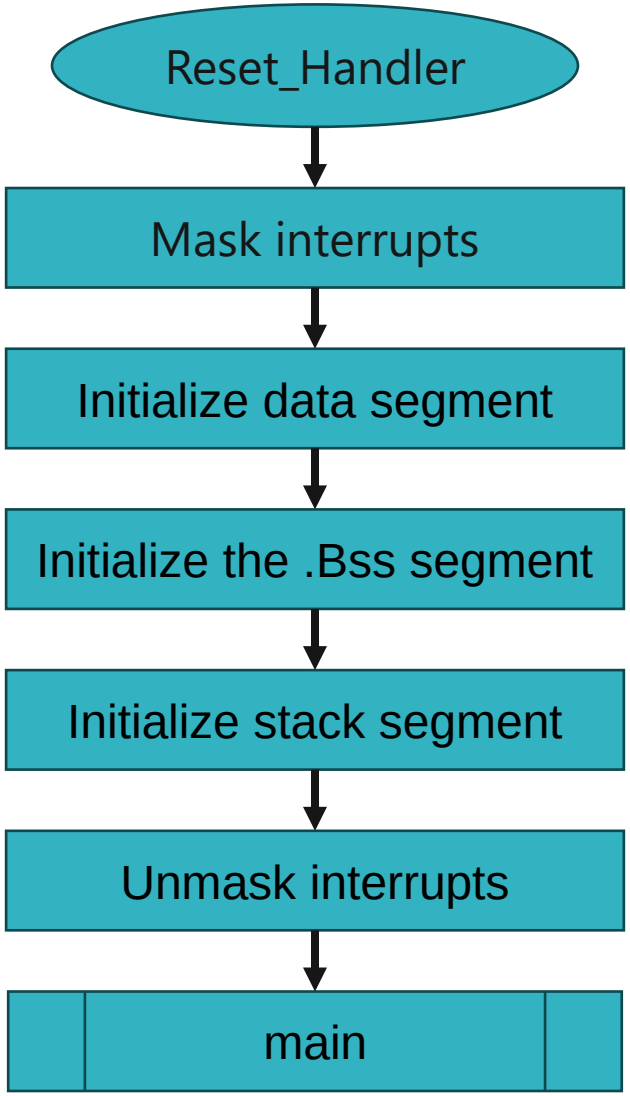
__attribute__((weak))

Variable attributes:

__attribute__((section("name")))

__attribute__((used))

6. Booting process



7. Inline assembly code

The compiler provides an inline assembler that enables you to write assembly code in your C or C++ source code.

```
__asm [volatile] ("assembly instruction");
```

Example:

```
__asm volatile ("LDR R1, =0x11");
```

```
__asm volatile ("CPSID I"); //Mask interrupts
```

```
__asm volatile ("CPSIE I"); //Unmask interrupts
```


Thank you